

PROGRAM 6 (Locally Weighted Regression)

```
import numpy as np
import matplotlib.pyplot as plt
def lwr(x, X, y, tau):
    w = np.diag(np.exp(-np.sum((X - x)**2, axis=1)/(2*tau**2)))
    theta = np.linalg.inv(X.T @ w @ X) @ X.T @ w @ y
    return x @ theta
np.random.seed(42)
X = np.linspace(0, 2*np.pi, 100)
y = np.sin(X) + 0.1*np.random.randn(100)
Xb = np.c_[np.ones(len(X)), X]
Xt = np.linspace(0, 2*np.pi, 200)
Xtb = np.c_[np.ones(len(Xt)), Xt]
tau = 0.5
yp = np.array([lwr(x, Xb, y, tau) for x in Xtb])
plt.figure(figsize=(10,6))
plt.scatter(X, y, color='red', label='Training Data', alpha=0.7)
plt.plot(Xt, yp, color='blue', label=f'LWR Fit (tau={tau})', linewidth=2)
plt.xlabel('X')
plt.ylabel('y')
plt.title('Locally Weighted Regression')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

PROGRAM 8 (Decision tree)

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)
print("Model Accuracy:", accuracy_score(y_test, clf.predict(X_test))*100)
print("Predicted Class for the new sample:",
      "Benign" if clf.predict([X_test[0]])[0] else "Malignant")
plt.figure(figsize=(12,8))
plot_tree(clf, filled=True,
          feature_names=data.feature_names,
          class_names=data.target_names)
plt.title("Decision Tree - Breast Cancer Dataset")
plt.show()
```

PROGRAM 9(Naive Bayesian)

```
from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt

data = fetch_olivetti_faces(shuffle=True, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.3, random_state=42
)
gnb = GaussianNB().fit(X_train, y_train)
y_pred = gnb.predict(X_test)
print(f'Accuracy: {accuracy_score(y_test, y_pred)*100:.2f}%')
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=1))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print(f'\nCross-validation accuracy: {cross_val_score(gnb, data.data, data.target,
cv=5).mean()*100:.2f}%')
fig, ax = plt.subplots(3, 5, figsize=(12,8))
for a, img, t, p in zip(ax.ravel(), X_test, y_test, y_pred):
    a.imshow(img.reshape(64,64), cmap='gray')
    a.set_title(f"True: {t}, Pred: {p}")
    a.axis('off')
plt.show()
```

PROGRAM 10(k-means clustering)

```
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt
X, y = load_breast_cancer(return_X_y=True)
X = StandardScaler().fit_transform(X)
kmeans = KMeans(n_clusters=2, random_state=42)
clusters = kmeans.fit_predict(X)
print("Confusion Matrix:")
print(confusion_matrix(y, clusters))
print("\nClassification Report:")
print(classification_report(y, clusters))
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
centers = pca.transform(kmeans.cluster_centers_)
plt.figure(figsize=(6,4))
plt.scatter(X_pca[:,0], X_pca[:,1], c=clusters, cmap='coolwarm')
plt.title("K-Means Clustering")
plt.show()
plt.figure(figsize=(6,4))
plt.scatter(X_pca[:,0], X_pca[:,1], c=y, cmap='coolwarm')
plt.title("True Labels")
plt.show()
plt.figure(figsize=(6,4))
plt.scatter(X_pca[:,0], X_pca[:,1], c=clusters, cmap='coolwarm')
plt.scatter(centers[:,0], centers[:,1], c='red', marker='X', s=200)
plt.title("K-Means Clustering with Centroids")
plt.show()
```

PROGRAM : 5 (KNN)

```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
data=np.random.rand(100)
train,test=data[:50],data[50:]
labels=["Class1" if x<=0.5 else "Class2" for x in train]
def knn(x,k):
    d=sorted((abs(x-train[i]),labels[i]) for i in range(50))
    return Counter(label for _,label in d[:k]).most_common(1)[0][0]
for k in [1,2,3,4,5,20,30]:
    result=[knn(x,k) for x in test]
    print(f"\nResults for k={k}")
    for i,label in enumerate(result,start=51):
        print(f"x{i}={test[i-51]:.4f}->{label}")
    c1=[test[i] for i in range(50) if result[i]=="Class1"]
    c2=[test[i] for i in range(50) if result[i]=="Class2"]
    plt.figure(figsize=(8,5))
    plt.scatter(train,[0]*50,c=['blue' if l=="Class1" else 'red' for l in labels],label='Training
Data')
    plt.scatter(c1,[1]*len(c1),c='blue',marker='x',label='Class1(Test)')
    plt.scatter(c2,[1]*len(c2),c='red',marker='x',label='Class2(Test)')
    plt.title(f'KNN Classification(k={k})')
    plt.xlabel('Data Values')
    plt.ylabel('Class')
    plt.legend()
    plt.grid(True)
    plt.show()
```

PROGRAM : 7

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, r2_score
```

Linear Regression

```
X,y=fetch_california_housing(as_frame=True).data[['AveRooms']],fetch_california_housing(as_frame=True).target
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
lr=LinearRegression().fit(X_train,y_train)
y_pred=lr.predict(X_test)
plt.scatter(X_test,y_test,label="Actual")
plt.plot(X_test,y_pred,'r',label="Predicted")
plt.title("Linear Regression")
plt.legend()
plt.show()
print("Linear Regression")
print("MSE:",mean_squared_error(y_test,y_pred))
print("R2 Score:",r2_score(y_test,y_pred))
```

Polynomial Regression

```
url="https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data"
cols=["mpg","cylinders","displacement","horsepower","weight","acceleration","model_year","origin","car_name"]

data=pd.read_csv(url,sep=r"\s+",names=cols,na_values="?").dropna()
X,y=data[['displacement']],data['mpg']
poly=make_pipeline(PolynomialFeatures(2),LinearRegression())
poly.fit(X,y)
y_pred=poly.predict(X)
plt.scatter(X,y,c='blue',label='Actual')
plt.scatter(X,y_pred,c='red',label='Predicted')
plt.title("Polynomial Regression")
plt.legend()
plt.show()
print("\nPolynomial Regression")
print("MSE:",mean_squared_error(y,y_pred))
print("R2 Score:",r2_score(y,y_pred))
```

Program 1 (Histogram, Boxplot)

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
data = fetch_california_housing(as_frame=True)
housing_df = data.frame
numerical_features = housing_df.select_dtypes(include=[np.number]).columns
plt.figure(figsize=(15, 10))
for i, feature in enumerate(numerical_features):
    plt.subplot(3, 3, i + 1)
    sns.histplot(housing_df[feature], kde=True, bins=30, color='blue')
    plt.title(f'Distribution of {feature}')
plt.tight_layout()
plt.show()
plt.figure(figsize=(15, 10))
for i, feature in enumerate(numerical_features):
    plt.subplot(3, 3, i + 1)
    sns.boxplot(x=housing_df[feature], color='orange')
    plt.title(f'Box Plot of {feature}')
plt.tight_layout()
plt.show()
print("Outliers Detection:")
outliers_summary = {}
for feature in numerical_features:
    Q1 = housing_df[feature].quantile(0.25)
    Q3 = housing_df[feature].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = housing_df[(housing_df[feature] < lower_bound) | (housing_df[feature] >
upper_bound)]
    outliers_summary[feature] = len(outliers)
    print(f'{feature}: {len(outliers)} outliers')
print("\nDataset Summary:")
print(housing_df.describe())
```

Program 2(Correlation Matrix)

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
california_data = fetch_california_housing(as_frame=True)
data = california_data.frame
correlation_matrix = data.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f',
linewidths=0.5)
plt.title('Correlation Matrix of California Housing Features')
plt.show()
sns.pairplot(data, diag_kind='kde', plot_kws={'alpha': 0.5})
plt.suptitle('Pair Plot of California Housing Features', y=1.02)
plt.show()
```

Program 3 (PCA)

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
iris = load_iris()
X_pca = PCA(n_components=2).fit_transform(iris.data)
plt.figure(figsize=(8, 6))
colors = ['r', 'g', 'b']
for i in range(3):
    plt.scatter(
        X_pca[iris.target == i, 0],
        X_pca[iris.target == i, 1],
        color=colors[i],
        label=iris.target_names[i]
    )
plt.title('PCA on Iris Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.grid()
plt.show()
```

Program 4 (Find S Algo)

```
data = {
    'Sky': ['Sunny', 'Sunny', 'Rainy', 'Sunny'],
    'Temp': ['Warm', 'Warm', 'Cold', 'Warm'],
    'Humidity': ['Normal', 'High', 'High', 'High'],
    'Wind': ['Strong', 'Strong', 'Strong', 'Strong'],
    'Water': ['Warm', 'Warm', 'Warm', 'Cool'],
    'Forecast': ['Same', 'Same', 'Change', 'Change'],
    'Play': ['Yes', 'Yes', 'No', 'Yes']
}
df = pd.DataFrame(data)
attributes = df.columns[:-1]
target = df.columns[-1]
hypothesis = ['?'] * len(attributes)
for _, row in df.iterrows():
    if row[target].strip().lower() == 'yes':
        for i, val in enumerate(row[attributes]):
            if hypothesis[i] == '?':
                hypothesis[i] = val
            elif hypothesis[i] != val:
                hypothesis[i] = '?'
print("Final Hypothesis:")
print(hypothesis)
```